



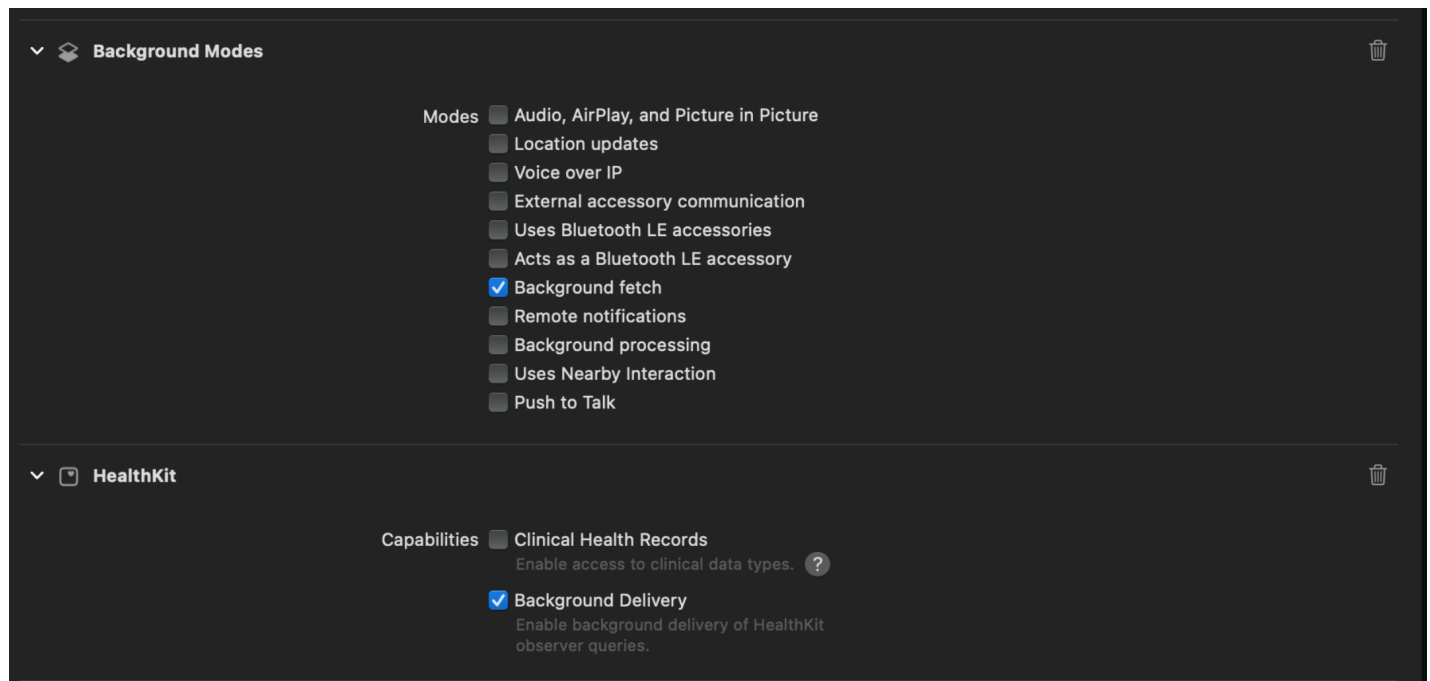
Configure Background

Setting background functions allows us to send new information as we receive notifications. This helps to resend that information to you through your webhook previously configured in the ROOK Portal.

Enable background sync of data for iOS

To configure background upload, follow the steps below:

- Add HealthKit to your project and enable background delivery.
- Add Background Modes and enable Background Fetch.



In the app delegate of your app add the `setBackListeners` method in `didFinishLaunchingWithOptions` function.

```
import UIKit
import Capacitor
import RookSDK
```

```
import RookAppleHealth

@UIApplicationMain
class AppDelegate: UIResponder, UIApplicationDelegate {

    var window: UIWindow?

    func application(_ application: UIApplication, didFinishLaunchingWithOptions
launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
        // Override point for customization after application launch.
        RookBackgroundSync.shared.setBackListeners()
        setupNotification()
        handleEvents()
        return true
    }

    ...
}
```

Then we need to enable or indicate to the SDK to stay listening for new notifications. We recommend enabling this in your dashboard to ensure that the user activates this function, like this:

```
/* eslint-disable react-hooks/exhaustive-deps */
import {
    IonButtons,
    IonContent,
    IonHeader,
    IonPage,
    IonTitle,
    IonToolbar,
    IonButton,
    IonBackButton,
    IonList,
    IonAlert,
    IonItem,
    IonDatetime,
    DatetimeChangeEventDetail,
} from "@ionic/react";
import "./Home.css";
import { RookAppleHealth } from "capacitor-rook-sdk";

const Dashboard: React.FC = () => {
    const [syncing, setSyncing] = useState(false);

    useEffect(() => {
        if (Platform.OS === "ios") {
```

```

    RookAppleHealth.enableBackgroundUpdates();
    RookAppleHealth.enableBackgroundEventsUpdates();
  }
}, []);

return (
  <IonPage>
    <IonHeader>
      <IonToolbar>
        <IonButtons slot="start">
          <IonBackButton defaultHref="/" />
        </IonButtons>
        <IonTitle>Dashboard</IonTitle>
      </IonToolbar>
    </IonHeader>
    <IonContent fullscreen>
      <IonButton onClick={goBack}>Go Back</IonButton>
    </IonContent>
  </IonPage>
);
};

export default Background;

```

Health Connect Background sync

- `scheduleHealthConnectBackground` Starts the background sync
- **Background Sync:** Syncs every hour without opening the app, recommended if your project supports Android 15 (SDK 35).
- **Continuous Upload:** Syncs every time you call it on foreground, recommended if your project is still on Android 14 (SDK 34) or lower.

`RookHealthConnect` allows to schedule an automatic syncs every hour in the background.

The following table describes `Background Sync` frequency of synchronization and historical range of each Health structure:

Health structure	Frequency	Historical range
Sleep Summary	Once per day	29 days in the past until yesterday
Body Summary	Once per day	29 days in the past until yesterday
Physical Summary	Once per day	29 days in the past until yesterday
Physical Activity	Once per hour	29 days in the past until today
Steps	Once per hour	Today
Calories	Once per hour	Today
Body metrics	Once per hour	Today
Heart rate	Once per hour	Today
Oxygenation	Once per hour	Today
Temperature	Once per hour	Today
Hydration	Once per hour	Today
Nutrition	Once per hour	Today
Blood pressure	Once per hour	Today
Blood glucose	Once per hour	Today

 INFO

`Background Sync` also requires a [user id](#), so it's possible that the first time nothing will happen, only after a user id is configured and all necessary permissions are granted the automatic sync will happen within the next hour.

**TIP**

It's a good practice to ask your users if they want to enable this behaviour, then save their preference in local storage and call `schedule` conditionally.

**INFO**

Once started `Background Sync` will attempt to sync all historic data, and will stop after it finishes or until the [Health Connect request quota](#) is exceeded.

Execution/Skip conditions

Each periodic execution of `Background Sync` will check for some requirements, and will skip the execution if one the following conditions is true:

- The device battery is low.
- The device storage is low.
- The device is not connected to the internet.
- The userID hasn't been [configured](#).
- The most recent request exceeded the [Health Connect request quota](#).
- The user hasn't granted Health Connect Permissions.
- The user hasn't granted Background Read Permissions.
- There is an error initializing the SDK.
- `cancelHealthConnectBackGround`: To cancel any scheduled background sync
- `checkBackgroundReadStatus`

Health Connect now supports full background data reads, but it has a few requirements, first is that the user MUST grant a new permission (`READ_HEALTH_DATA_IN_BACKGROUND`) and the second is that the user's device MUST have a Health Connect application version that supports background reads.

You can use the function `checkBackgroundReadStatus`

Health Connect request quota

To maintain optimal system stability and performance, Health Connect imposes rate limits on client connections to the Health Connect API.

It is important to understand that every data type in the ROOK SDK is constructed of multiple health variables like heart rate, step count, hydration, etc. When a Sleep Summary is extracted, multiple calls are made to the Health Connect API. While we have focused on optimizing and reducing the number of API calls required for each data type, it is still possible to reach the limit, especially when performing multiple extractions in a short period.

Depending on the sync type you choose, keep the following in mind:

Request quota when syncing data manually

- Extract summaries once daily: Since summaries collect health data from the previous day, there is no need to extract them more than once per day.
- Use what you already have: If you are extracting Physical Events, you do not need to extract Heart Rate Events (Physical) or Oxygenation Events (Physical) as these are already included in the PhysicalEvent object.
- Only sync the relevant health data for your use case: If you are not interested in individual events and only want to sync the summary of a specific date, use the `sync` functions in `RookSummaries`.
- If you have already reached the request quota, avoid calling any `sync` function for the next few hours to allow your quota to recover.

Request quota when syncing data automatically

`scheduleYesterdaySync` addresses practically all issues and limitations of Health Connect. When the quota is reached, all pending syncs are canceled, and a recovery timestamp is created. Pending syncs will not resume until the user reopens the app after a few hours.

Enable continuous upload of data for Android

To automatically sync health data, use the `scheduleYesterdaySync` function during your app's initialization phase. We highly recommend checking and saving whether the user has explicitly given you permission to sync data automatically

First we need to request permissions, like we see in the first steps of [requesting android background permissions](#).

`doOnEnd` available configurations:

- `oldest`: After syncing yesterday's data, subsequent syncs should start from the 29th day until the 2nd day (the day before yesterday).
- `latest`: After syncing yesterday's data, subsequent syncs should start from the 2nd day (the day before yesterday) until the 29th.
- `nothing`: Only sync yesterday's data.

```
import React, { useEffect } from "react";
import { RookHealthConnect } from "capacitor-rook-sdk";

const Component: React.FC = () => {
  useEffect(() => {
    tryToEnableYesterdaySync();
  }, []);

  const tryToEnableYesterdaySync = async () => {
    console.log("Attempting to enable yesterday sync...");

    try {
      await RookHealthConnect.scheduleAndroidYesterdaySync("oldest");
    } catch (error) {
      console.error("Error:", error);
    }
  };

  return <IonPage>{/* Your UI components here */}</IonPage>;
};

export default MyComponent;
```

RookYesterdaySyncPermissions

`scheduleYesterdaySync` Requires 2 types of permissions

- Android
 - POST_NOTIFICATIONS
 - FOREGROUND_SERVICE

- FOREGROUND_SERVICE_HEALTH
- ACTIVITY_RECOGNITION
- Health Connect
 - SLEEP
 - PHYSICAL
 - BODY

To request or check both types of permissions, you have to **request permissions** for Health Connect with `RookPermissions.requestPermissions()` to access Health Connect data and `RookPermissions.requestAndroidBackgroundPermissions()` to access background services.

Customizing the foreground service notification

To sync health data automatically, a Foreground Service is used. This service requires a notification to be displayed until the synchronization finishes.

To use your own resources, you need to reference them in the **AndroidManifest.xml** file:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <application>
    <meta-data
      android:name="io.tryrook.service.notification.SYNC_ICON"
      android:resource="@drawable/my_custom_icon"/>

    <meta-data
      android:name="io.tryrook.service.notification.SYNC_TITLE"
      android:resource="@string/my_custom_title"/>

    <meta-data
      android:name="io.tryrook.service.notification.SYNC_CONTENT"
      android:resource="@string/my_custom_content"/>
  </application>
</manifest>
```



Starting with Android 13 (SDK 33), this notification can be dismissed without finishing the service associated with it. The service will then be displayed in the active apps section (this may vary depending on the device brand).

Launch/Stop conditions

`scheduleYesterdaySync` won't start or will stop if any of the following conditions are met:

- The device battery is low.
- The device storage is low.
- The device is not connected to the internet.
- The user hasn't granted Android permissions (POST_NOTIFICATIONS, FOREGROUND_SERVICE, FOREGROUND_SERVICE_HEALTH).
- The device has previously exceeded the [Health Connect request quota](#) and the recovery timestamp hasn't been met.
- The user hasn't granted Health Connect permissions (SLEEP, PHYSICAL, BODY).
- The most recent request exceeded the [Health Connect request quota](#).
- The userID hasn't been [configured](#).
- There is an error initializing the SDK.

Samsung Health Background sync

`Background Sync` allows to schedule an automatic syncs every hour in the background.

Each time background sync is triggered the following operations will happen:

1. Sync today events:
 - i. Activity
 - ii. Steps
 - iii. Calories
 - iv. Body Metrics
 - v. Heart Rate
 - vi. Oxygenation
 - vii. Hydration

viii. Nutrition

ix. Blood Pressure

x. Blood Glucose

2. Sync today summaries

i. Sleep

- If at least 4 hours have passed since the last successful sync
- If the "new" data is equal to the previous sync it won't be synced again.

3. Sync yesterday summaries

i. Sleep

- If at least 4 hours have passed since the last successful sync
- If the "new" data is equal to the previous sync it won't be synced again.

ii. Physical

- If at least 4 hours have passed since the last successful sync
- If the "new" data is equal to the previous sync it won't be synced again.

iii. Body

- If at least 4 hours have passed since the last successful sync
- If the "new" data is equal to the previous sync it won't be synced again.

4. Sync historic data

i. Activity events

- If not synced yet 29 days, starting from yesterday

ii. Sleep summary

- If not synced yet 29 days, starting from yesterday

iii. Physical summary

- If not synced yet 29 days, starting from yesterday

iv. Body summary

- If not synced yet 29 days, starting from yesterday

To use `Background Sync` you will need:

- Samsung Health permissions: Go to the main [Samsung Health availability & permissions](#) section to see the implementation.

! INFO

`Background Sync` also requires a [user id](#), so it's possible that the first time nothing will happen, only after a user id is configured and all necessary permissions are granted the automatic sync

will happen within the next hour.

To schedule a background sync for every hour call `enableBackgroundUpdates` in `RookSamsungHealth` module:

```
const enableBackgroundUpdates: (): Promise<BoolResult>
```

To cancel any scheduled background sync call `cancel`:

```
const disableBackgroundUpdates: (): Promise<BoolResult>
```

If you want to check the current state of `Background Sync` call:

```
const isBackgroundUpdatesEnable: (): Promise<BoolResult>
```